

SIMATS ENGINEERING
ASSESSMENT TOOL 4 - LOW
(CODING CHALLENGE)

COURSE CODE: ITA04

COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO2: *Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic. (BL3, BL4)*

Assessment Tool Weightage: 15%

QUESTIONS: 20

Total Marks: 100

ANNEXURE A

SCENARIO BASED QUESTIONS QUESTIONS

Code of Conduct:

*I **S. V. K. Sharon Surya (192324246)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

SCENARIO BASED QUESTIONS QUESTIONS

1. A data analyst wants to create a function that filters a numeric vector. The function must multiply all positive numbers by 2. If a number is negative or zero, it must remain unchanged. Which control structure is the most efficient and standard way to handle this vectorised operation in R?

- A) A for loop combined with an if-else statement.
- B) A while loop that increments an index vector.
- C) The vectorised ifelse() function.
- D) A recursive function that processes elements one by one.

2. A developer writes a data cleanup function with the signature `clean_data <- function(df, remove_na = TRUE, threshold = 0.05)`. During a production run, the colleague calls the function using `clean_data(my_df, 0.01)`. What will be the behavior of the arguments inside the function?

- A) `remove_na` becomes 0.01 and `threshold` uses its default value of 0.05.
- B) R throws a positional argument error because `remove_na` expects a logical value.
- C) `remove_na` retains TRUE and `threshold` is assigned 0.01.
- D) Both arguments are ignored because the positions do not match the data types.

3. Consider the following R function designed to process a vector:

R

```
process_vector <- function(x) {  
  if (any(is.na(x))) {  
    return(NULL)  
  }  
  x * 5  
}
```

What does this function return if it is called with `process_vector(c(1, 2, 3))`?

- A) It returns NULL because `return()` was not called at the end of the function.
- B) It returns the vector `c(5, 10, 15)` via implicit return.
- C) It throws a syntax error because a function cannot have both explicit and implicit returns.
- D) It returns nothing (invisible) because the last line lacks an explicit `return()` wrapper.

4. A bioinformatics script needs to return the statistical summaries, the original cleaned data frame, and a diagnostic plot from a single function execution. Since R functions can only return a single object, what is the best way to accomplish this?

- A) Return a character vector containing the paths to where these objects are saved in the global environment.
- B) Use a pointer to pass the address of the data frame and update it externally.
- C) Combine the data frame, summaries, and plot into a named list object and return the list.
- D) Assign the variables directly to the global environment using the `<<-` operator inside the function without returning anything.

5. A Python programmer is learning R and wants to optimize memory usage when working with large matrices. They attempt to pass a matrix to a function to modify its cells directly in-place, expecting pointer-like behavior. Why will this approach fail to mutate the original matrix outside the function scope?

- A) R requires all matrices to be converted to lists before pointers can reference them.
- B) R passes arguments using a copy-on-write mechanism and does not expose pointers to manipulate memory addresses directly.
- C) The user forgot to activate the memory pointer tracking system using `options(pointers = TRUE)`.
- D) Functions in R can only modify data frames in-place, not matrices.

6. Review the following snippet of code executing a loop:

```
R
val <- 1
while(val < 10) {
  val <- val * 2
  if(val == 4) next
  print(val)
}
```

What is the exact output printed to the console?

- A) 2, 4, 8, 16
- B) 2, 8, 16
- C) 2, 4, 8
- D) 2, 8

7. An engineer needs to evaluate whether a patient's lab results meet two conditions simultaneously: the glucose level must be over 100, and the systolic blood pressure must be over 140. They have two vectors representing a population. Which operators should they use to evaluate the entire population at once, ensuring vectorised logical evaluation?

- A) `&&` and `||`
- B) `&` and `|`
- C) `AND` and `OR`
- D) `%in%` and `==`

8. In R, "functions are first-class objects." A team wants to leverage this architecture to build a flexible modeling pipeline. Which of the following scenarios best demonstrates this concept?

- A) Passing a mathematical function like `mean` or `sd` as an argument into another function to change summary behaviors dynamically.
- B) Using a `for` loop to explicitly force a function to run on multiple CPU cores sequentially.
- C) Defining a function inside a spreadsheet cell to calculate column balances automatically.

D) Ensuring that a function can never be modified or overwritten once it is saved to a package directory.

9. A developer writes a recursive function to calculate the nth Fibonacci number:

R

```
fib <- function(n) {  
  if (n <= 1) return(n)  
  return(fib(n - 1) + fib(n - 2))  
}
```

If this function is called with a highly deep nesting level (e.g., fib(50000)), what risk does the developer face in standard R?

- A) The function will automatically convert the result into a complex matrix, losing precision.
- B) R will trigger a "stack overflow" error because it has a finite expression evaluation limit regulated by options(expressions).
- C) R will bypass the condition and run an infinite loop because recursion forces variables to become pointers.
- D) The function will switch to a vectorised while loop automatically, changing the expected output format.

10. A script contains an expression that checks missing values: if (x == NA) { print("Missing") }. When run with a single integer value x <- 5, what will be the result of this conditional statement?

- A) It will print "Missing" because NA acts as a wildcard match for any integer.
- B) It will evaluate to FALSE and safely skip the print statement.
- C) It will result in an error or warning: argument is of length zero.
- D) It will result in missing value where TRUE/FALSE needed because x == NA outputs NA, not a logical TRUE or FALSE.

11. A developer is writing a data intake function and wants a logical argument called verbose to default to FALSE if the user leaves it out. Which function signature achieves this correctly in R?

- A) read_data <- function(df, verbose = FALSE)
- B) read_data <- function(df, verbose <- FALSE)
- C) read_data <- function(df, verbose(FALSE))
- D) read_data <- function(df, verbose == FALSE)

12. You run the following block of code to find the first square number greater than 50:

R

```
x <- 1  
repeat {  
  if (x^2 > 50) {  
    print(x^2)  
    _____  
  }  
  x <- x + 1
```

```
}
```

Which keyword must fill the blank to prevent an infinite loop?

- A) next
- B) return
- C) break
- D) stop

13. A colleague passes a vector `v <- c(TRUE, FALSE, TRUE)` into an if statement: `if (v) { print("Valid") }`. What will R do when executing this script?

- A) It evaluates the first element TRUE, prints "Valid", and throws a warning about a condition length > 1.
- B) It automatically applies an implicit `all()` check and skips the print statement.
- C) It loops through the condition three separate times automatically.
- D) It throws a fatal execution error immediately without reading the block.

14. An R script tracks inventory. It must execute specific code when status matches exactly "Low", "Critical", or "Out". Instead of nesting multiple if-else blocks, which native R structure is best suited for matching a single character string against multiple options?

- A) A nested while loop
- B) The `switch()` function
- C) A vectorised `ifelse()` block
- D) The `%in%` operator inside a repeat loop

15. Consider this function that calculates a statistical summary:

```
R
analyze <- function(data) {
  res <- summary(data)
  # No return statement here
}
```

If a user runs `output <- analyze(c(1, 5, 10))`, what will the variable output contain?

- A) It will contain NULL.
- B) It will contain the summary object evaluated on the last line.
- C) It will be empty, and R will throw an "undefined return value" error.
- D) It will contain a logical TRUE indicating the function finished successfully.

16. A programmer attempts to write a recursive function to calculate the factorial of a number:

```
R
fact <- function(n) {
  return(n * fact(n - 1))
}
```

What critical component of a recursive function is missing from this snippet, causing an infinite loop?

- A) The explicit assignment operator
- B) A vectorised loop wrapper
- C) A base case condition to halt execution
- D) A pointer conversion mechanism

17. An analyst wants to test if a vector x contains *at least one* negative value. Which logical expression will evaluate the entire vector and return a single TRUE or FALSE value?

- A) $x < 0$
- B) `all(x < 0)`
- C) `any(x < 0)`
- D) $x \&\& 0$

18. You are reviewing code where a complex matrix calculation happens inside a function. The previous developer ended the function with `return(matrix_result)`. How would removing the explicit `return()` wrapper alter the function's output?

- A) The function would return NULL instead of the matrix.
- B) The function would print the matrix to the console but return nothing to the variable.
- C) The function behavior would not change; it still returns the matrix implicitly.
- D) R would crash because matrix objects require explicit return handling.

19. A data pipeline uses the expression `result <- A & B`. If A is `c(TRUE, FALSE)` and B is `c(TRUE, TRUE)`, what is the value of result?

- A) TRUE
- B) `c(TRUE, FALSE)`
- C) `c(TRUE, TRUE)`
- D) FALSE

20. An engineer writes a function that needs to return both a vector of coefficients and a diagnostic data frame. They want to make sure the user can easily extract these components by name using the `$` operator. What object should the function return?

- A) A matrix
- B) A multi-dimensional array
- C) A named list
- D) An unlisted character vector

21. Because functions in R are first-class objects, you can store them inside a list.

Review the following snippet:

R

```
math_ops <- list(add_one = function(x) x + 1, double = function(x) x * 2)
```

Use code with caution.

How do you invoke the second function in this list to process the number 5?

- A) `math_ops$double(5)`

- B) `math_ops[] * 5`
- C) `call(math_ops, double, 5)`
- D) `math_ops(2, 5)`

22. A script defines a simple loop:

```
R
for (i in 1:3) {
  if(i == 2) next
  print(i)
}
```

What is the exact output displayed in the console?

- A) 1, 2, 3
- B) 1, 3
- C) 1
- D) 3

23. A user wants to compare two numeric values x and y while completely avoiding short-circuiting behavior. They want to ensure both sides of the logical expression are evaluated every time. Which operator should they choose?

- A) `&&`
- B) `&`
- C) `||`
- D) `and`

24. A data cleaning function relies on a default threshold: `process <- function(val, cut = 10)`. If a developer calls this function using `process(5, cut = NULL)`, what value will the argument cut hold during execution?

- A) It falls back to its default value of 10.
- B) It evaluates to `NULL`, overriding the default value.
- C) R throws an error because `NULL` cannot be assigned to an argument with a default.
- D) It evaluates to `NA`.

25. An engineer coming from a C++ background wants to speed up an R script by passing a data frame pointer to a function. They want the function to add a column directly to the original global data frame without copying it. Why is this unachievable in standard R?

- A) Data frames can only be modified inside a while loop, not inside custom functions.
- B) R does not feature user-exposed pointer variables; object adjustments rely on copy-on-write or explicit reassignment.
- C) Global variables are completely locked and invisible inside custom functions.
- D) R functions can only return primitive atomic vectors, not structured tables.

26. What is the value of the logical expression `5 > 3 | 10 < 2` in R?

- A) `TRUE`
- B) `FALSE`

- C) NA
- D) NULL

27. Examine the following loop construct:

```
R
count <- 1
while(count < 4) {
  count <- count + 1
}
print(count)
```

What value is printed to the console when the loop finishes?

- A) 3
- B) 4
- C) 5
- D) 2

28. A function needs to terminate early and return a specific message if an input vector is empty. In which scenario is using an explicit return() statement highly recommended over an implicit return?

- A) At the very last line of the function block.
- B) Inside a conditional guard clause at the beginning of the function to exit early.
- C) Inside a standard variable assignment block.
- D) R functions never support explicit returns; they are always implicit.

29. A script evaluates an expression: res <- TRUE && FALSE. What is the value of res?

- A) TRUE
- B) FALSE
- C) c(TRUE, FALSE)
- D) NA

30. A team is building an optimization loop. They want a loop structure that is guaranteed to run the code block *at least once* before checking the conditional exit criteria. Which structure fits this requirement best?

- A) A standard for loop
- B) A while loop
- C) A repeat loop containing a conditional break at the end
- D) A recursive function call

31. A script runs the following command: val <- (TRUE | NA). What is the value of val?

- A) NA
- B) FALSE
- C) TRUE
- D) Error: logical operator cannot handle missing values

32. A package developer writes a function where one argument depends on another:

R

```
scale_data <- function(x, center = mean(x)) {  
  x - center  
}
```

If a user runs `scale_data(c(10, 20, 30))`, how does R evaluate the default argument `center`?

- A) R throws an error because `x` is not yet defined when the function signature is parsed.
- B) R uses lazy evaluation, computing `mean(x)` only when `center` is explicitly referenced inside the function body.
- C) R sets `center` to `NULL` automatically because it cannot look ahead.
- D) R searches the global environment for a variable named `x` before checking the function input.

33. Which arithmetic operator is used in R to calculate the remainder of a division (modulo operation)?

- A) `%/%`
- B) `mod`
- C) `%`
- D) `%%`

34. What happens if you try to assign a function to a new variable name, such as `my_log <- log`?

- A) R executes the function immediately and saves the numerical result.
- B) R throws a duplicate assignment syntax error.
- C) `my_log` becomes an alias for the `log` function object and can be called using `my_log(10)`.
- D) The original `log` function is deleted from system memory.

35. You want to execute a block of code only if a score is between 80 and 90 inclusive. Which conditional statement is written correctly for a single scalar value?

- A) `if (80 <= score <= 90)`
- B) `if (score >= 80 & score <= 90)`
- C) `if (score >= 80 && score <= 90)`
- D) `if (score >= 80 AND score <= 90)`

36. An analyst wants to write a function that calculates the weighted average of a vector. If the user does not provide weights, the function should automatically default to calculating the standard unweighted arithmetic mean. Which function definition correctly implements this option?

- A) `calculate_average <- function(x, weights = rep(1, length(x)))`
- B) `calculate_average <- function(x, weights <- rep(1, length(x)))`
- C) `calculate_average <- function(x, weights == mean(x))`
- D) `calculate_average <- function(x, weights = default)`

37. Review the following code snippet evaluating a logical state:

R

```
x <- c(10, 20, 30)
```

```
y <- c(5, 25, 15)
```

```
res <- (x > 15) && (y > 20)
```

What is the final exact value stored inside the scalar variable res?

A) FALSE

B) TRUE

C) c(FALSE, TRUE, FALSE)

D) NA

38. A researcher is building a structural validation function that tests clean dataset dimensions. Under what specific architectural circumstance must the developer explicitly invoke the return() function?

A) When outputting a multi-dimensional array or a nested list structure at the end of the script.

B) Inside an early-exit conditional block to interrupt function execution before hitting the terminal line.

C) When the output object contains missing numeric values or logical NA elements.

D) R functions strictly demand explicit returns for complex matrices and never support implicit output structures.

39. A system engineer is migrating code from an environment where functions are strictly declarative subroutines to R. They want to store three different metrics models inside an R object and pass that entire container into a processing engine. Why does R allow functions to be handled as elements inside data collections?

A) Because R converts all functional blocks into text strings before executing them.

B) Because R lacks pointers and treats code statements as raw data tables instead.

C) Because functions in R are first-class objects, meaning they can be treated like any other variable or value type.

D) Because functions are automatically mapped to global memory slots when defined.

40. Examine the following tracking loop designed to locate a specific threshold condition:

R

```
val <- 2
```

```
while (val < 7) {
```

```
  val <- val + 2
```

```
  if (val == 6) break}
```

```
print(val)
```

What exact final value is printed to the R console when the control structure exits?

A) 4

B) 6

C) 8

D) 2

Answers:

1. C) ifelse()

ifelse() is vectorised, so it can process all numbers at once without a loop.

2. A) remove_na becomes 0.01

R uses **positional arguments** when no argument name is given. So 0.01 goes to remove_na.

3. B) c(5, 10, 15)

There are no NA values, so $x * 5$ is executed. R returns the last expression automatically.

4. C) Named list

A list can store different objects such as data frames, summaries, and plots together.

5. B) Copy-on-write mechanism

R does not normally provide pointer-based modification like C/C++. Objects are handled using R's copy-on-modify semantics.

6. B) 2, 8, 16

Values are:

- $1 \times 2 = 2 \rightarrow \text{print}$
- $2 \times 2 = 4 \rightarrow \text{next, so don't print}$
- $4 \times 2 = 8 \rightarrow \text{print}$
- $8 \times 2 = 16 \rightarrow \text{print}$

7. B) & and |

& and | work element-by-element on vectors.

8. A) Passing a function as an argument

Functions are first-class objects, so they can be stored or passed to other functions.

9. B) Stack/expression limit risk

Very deep recursion can exceed R's evaluation/stack limits and cause an error.

10. D) NA

$\text{NA} == \text{NA}$ does not produce TRUE; comparisons involving NA generally produce NA. Use `is.na(x)` to test missing values.

11. A) `read_data <- function(df, verbose = FALSE)`

The `=` is used to provide a default argument value.

12. C) `break`

`break` immediately stops the repeat loop.

13. D) It throws an error

`if()` expects a **single TRUE/FALSE value**. A vector such as `c(TRUE, FALSE, TRUE)` causes a condition-length error.

14. B) `switch()`

`switch()` is useful for selecting one option from several possible values.

15. B) Summary object

R automatically returns the value of the last expression in a function.

16. C) Base case condition

Recursion needs a stopping condition. Without it, `fact()` keeps calling itself.

17. C) `any(x < 0)`

`any()` returns TRUE if **at least one** element satisfies the condition.

18. C) Behavior does not change

R can return the last expression implicitly, so:

```
return(matrix_result)
```

and simply:

```
matrix_result
```

give the same result at the end.

19. B) `c(TRUE, FALSE)`

& compares corresponding elements:

- `TRUE & TRUE → TRUE`
- `FALSE & TRUE → FALSE`

20. C) Named list

A named list allows easy access using \$, such as:

```
result$coefficients  
result$data
```

21. A) `math_ops$double(5)`

`double` is the name of the second function in the list.

22. B) 1, 3

When `i == 2`, `next` skips the `print()` statement.

23. B) `&`

`&` evaluates both sides element-by-element. `&&` is a short-circuit operator.

24. B) `NULL`

Explicitly providing `NULL` overrides the default value of 10.

25. B) R does not feature user-exposed pointer variables

R does not provide normal C++-style pointers for directly modifying an object in this way.

26. A) `TRUE`

```
5 > 3
```

is `TRUE`, and:

```
10 < 2
```

is `FALSE`.

Therefore:

```
TRUE | FALSE
```

is `TRUE`.

27. B) 4

The loop increases count:

```
1 → 2 → 3 → 4
```

Then `count < 4` becomes `false`.

28. B) Inside an early-exit conditional

`return()` is especially useful when you need to stop the function early.

Example:

```
if (length(x) == 0) {  
  return("Empty")  
}
```

29. B) FALSE

`TRUE && FALSE`

results in FALSE.

30. C) repeat with break

A repeat loop runs at least once and can be stopped using `break`.

31. C) TRUE

In R:

`TRUE | NA`

is TRUE because `TRUE OR anything` is TRUE.

32. B) Lazy evaluation

R calculates `mean(x)` when `center` is actually needed inside the function.

33. D) `%%`

`%%` gives the remainder.

Example:

`10 %% 3`

Result:

1

34. C) `my_log` becomes an alias for the function

`my_log <- log`

Then:

`my_log(10)`

works like:

`log(10)`

35. C) `if (score >= 80 && score <= 90)`

`&&` is suitable when checking two conditions for a single value.

36. A) `weights = rep(1, length(x))`

This provides default weights of 1 for every element.

37. A) FALSE

`&&` uses only the first element of each logical vector:

`x > 15`

first element $\rightarrow$ FALSE

So:

`FALSE && TRUE`

is FALSE.

38. B) Inside an early-exit conditional

Explicit `return()` is useful when the function needs to stop before reaching the final line.

39. C) Functions are first-class objects

In R, functions can be stored in variables, lists, and passed as arguments.

40. B) 6

The loop goes:

$2 \rightarrow 4 \rightarrow 6$

When `val == 6`, `break` stops the loop.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification